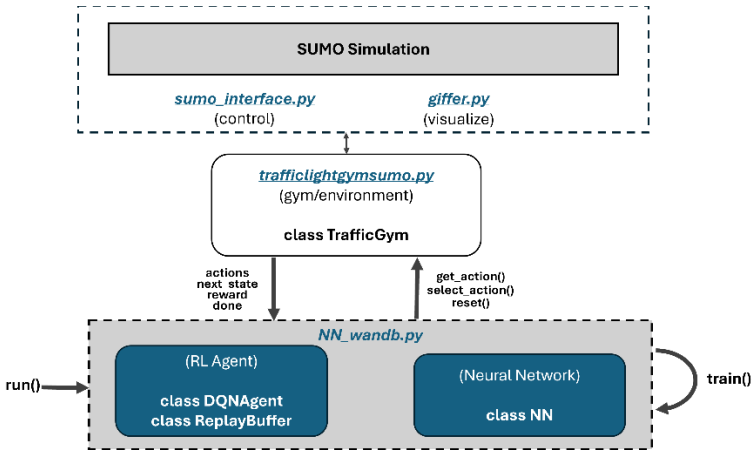
Learning Agent Report

Overview

This report describes our implementation for the learning agent and Deep Q-network (DQN). The agent learns to minimise queue lengths and waiting times by selecting optimal traffic light sequences. This report will provide a code explanation for the DQN agent, talk about the external code libraries used and reflections on what we have learnt from this milestone.

Code Explanation



Project Structure

The figure above shows the project structure. The **DQNAgent** class is the main controller, which enables deep Q-learning. It implements a **ReplayBuffer,** which stores 100,000 experiences for learning.

The training starts with the ***run*** function, resetting the environment for each new episode. We select the action using an epsilon-greedy policy (implemented in function **select_*action***). Our initial iterations select a new action every second, which is not realistic. Instead, **get_action** is implemented internally, which calls ***select_action*** to select and apply the next action after 10 seconds. After this, the **ReplayBuffer** is updated with the last action, states and reward. ***train*** is called which implements the core DQN algorithm using the Bellman equation. The current Q-values are computed from the policy network for the actions taken while next Q-values are taken from the target network to find the maximum Q-value for each next state. Finally, the target Q-values are calculated using the reward plus the next discounted Q-value. The loss is taken as the mean squared error between the current and target Q-value. To ensure training stability, Q-value clipping to a range of –1000 to 1000 was implemented to prevent exploding Q values experienced initially. Gradients are also clipped to a maximum of 1.0 to prevent exploding gradients.

Statistics of each episoide are logged into the Weights and Biases (**wandb** library) platform for the monitoring and debugging of the learning agent. Key statistics monitored include:

- Average episode reward over the past 100 episodes to monitor the reward trend

- Minimum and maximum episode rewards to ensure that the best and worst episodes are improving over time
- Minimum and maximum of each reward component (delta queue-length and penalty for long waiting times) to tune the weights of each component for proper reward function scaling
- Actions taken to monitor if the agent is choosing logical actions
- Loss of each step taken to monitor if it is being minimised through gradient descent and whether the agent is learning optimal policies

Finally, ***evaluate*** is performed every 50 episodes using a pure exploitation policy on the trained policy parameters performed on a separate environment. This provides a measure of the true learned policy without exploration noise during training.

Existing Code/Libraries

For the learning agent, **PyTorch** was selected as the library to be used for the learning agent. PyTorch was chosen for its simplicity, flexbility and strong community support for debugging purposes. **NumPy** was also used for its simplified and optimized matrix handling. Finally, we used Weights & Biases (**wandb**) for experiment tracking. Wandb has a simple python package to enable easy implementation within our code, a customisable dashboard to visualise and compare runs across various metrics as well as a collaborative interface where teams can view each group member's runs and training performance, which was extremely useful for tracking the learning of our agent.

Reflections

Implementing a learning agent was initially extremely challenging due to our unfamiliarity with methods to track learning performance. It was difficult interpreting the metrics and where or how to begin debugging our agent. However, through meticulous variable tuning and rigorous ablation studies, we were able to figure out various factors in our initial code that were stopping our agent from learning an effective policy.

Our initial proposal assumed that the agent would select an action every second, which was incredibly unrealistic and caused major issues for the agent. Another challenge was determining an effective reward function for learning. The delta queue length component of the reward was giving relatively small values, but our penalty for long wait for cars were large negative values which accumulated every step, causing incredibly negative rewards. One key learning point is that designing and curating an effective reward function is incredibly paramount in deep reinforcement learning.

Tuning of the hyperparameters also proved to be a challenge. There are many hyperparameters and applying what was learned from lectures to an actual learning agent was harder that we thought. We had to learn and understand the effects of each hyperparameter to stabilise our agent.

Future improvement for the project includes testing different variations of DQN agents such as duelling and double DQN, which we speculate would improve and speed up training. We would also like to compare our agent with conventional traffic signal control algorithms. This would allow us to validate the effectiveness of our agent against a traditional real-world solution.

AI Tool Declaration:

I (Lam Kai Yi) used Github Copilot to unify the gym, agent, sumo and training configurations as there were many settings all over the document. Unifying the configuration allowed us to be able to track all the configurations we used during testing. I also used Claude AI to inform me of the useful metrics to track the performance for a DQN agent. However, the code was self-implemented and tested to identify which metrics were useful for the project. I am responsible for the content and quality of the submitted work.

I (Goh Chian Kai) used Claude AI to obtain information on the different variants of DQN agent for implementation in python. I am responsible for the content and quality of the submitted work.